

Comparing Global Instruction Files for AI Coding Agents:

An Empirical Benchmark of AGENTS.md Variants

Sathvik A R PES University
Bengaluru, India
arsathvik48@gmail.com

Abstract—Large language model (LLM)-based coding agents are increasingly autonomous, yet their behavior is heavily shaped by natural-language instruction files placed in the project root. This paper presents an empirical comparison of five candidate AGENTS.md instruction files—ranging from no instructions (control) to multi-agent orchestration protocols—using the OpenCode coding agent across 25 self-contained Python tasks and four thematic tracks. Each variant is evaluated under identical headless conditions with a fixed model (opencode-go/ox-alpha-free) and a 25-minute time budget. Round 1 (bug-fix suite, 20 tasks, 174 unit tests) yields perfect accuracy for all five variants; only wall-clock time differs, with the test-driven development variant completing fastest at 5.9 minutes. Round 2 introduces four thematic tracks—spec-to-code, refactor-under-constraints, SQL analytics, and API client—with 177 hidden correctness tests, 38 structural checks, and 15 exact-match SQL questions. Only the orchestration-heavy variant achieves 100% on the spec-to-code track, the sole track that breaks saturation; all other variants score 93% or below, with the test-driven variant at 36% and the tiered-orchestration variant at 0.56% due to provider-level stream drops. Across all tracks, the orchestration-heavy variant averages 100.0%, followed by the control (99.86%), concise (98.31%), test-driven (84.04%), and tiered-orchestration (75.14%). We find that instruction-file complexity trades off against speed on narrow tasks but pays dividends on breadth-under-deadline scenarios, and that provider reliability is a confounding variable that can dominate instruction quality for tiered protocols.

Index Terms—AI coding agents, LLM instruction engineering, benchmarking, AGENTS.md, OpenCode, test-driven development, multi-agent orchestration.

I. INTRODUCTION

The emergence of LLM-based coding agents—tools that autonomously read, write, and debug source code—has shifted the software-engineering workflow from “human writes code” to “human instructs agent, agent writes code.” Frameworks such as Cursor [1], Aider [2], and OpenCode [3] now operate headlessly in continuous-integration pipelines, executing multi-file refactors, generating test suites, and iterating on failing assertions.

A critical but under-studied component of these agents is the *instruction file*: a natural-language document placed in the project root that biases the agent’s decision-making. OpenCode uses AGENTS.md, Claude Code uses CLAUDE.md, and similar conventions exist across platforms. Despite their ubiquity, no systematic comparison of instruction-file designs has been published.

This paper addresses that gap with two rounds of controlled experiments:

- 1) **Round 1** (Section IV): a bug-fix suite of 20 Python tasks with 174 unit tests, measuring accuracy and speed under five instruction variants.
- 2) **Round 2** (Section V): four thematic tracks (spec-to-code, refactor, SQL analytics, API client) with 215 hidden tests, measuring accuracy, speed, and failure modes under the same five variants.

Our contributions are:

- The first empirical comparison of AGENTS.md instruction-file designs.
- Evidence that instruction complexity trades off against speed on narrow tasks but improves breadth-under-deadline performance.
- Identification of provider reliability as a confounding variable that can dominate instruction quality.
- Three reusable OpenCode specialist agents (arena-runner, arena-analyst, suite-author) distilled from the experiment’s findings, released as open-source artifacts.

II. RELATED WORK

A. Prompt Engineering for Code Generation

Prompt engineering for code generation has focused on task-level prompts (e.g., few-shot examples, chain-of-thought) rather than project-level instruction files. GitHub Copilot [8] and CodeLlama [7] optimize the prompt format for single-file generation but do not address multi-file, multi-session workflows where project-level instructions dominate.

B. Benchmarking Coding Agents

SWE-bench [4] evaluates agents on real GitHub issues; HumanEval [5] measures function-level code generation; MBPP [6] tests basic programming problems. None of these benchmarks varies the instruction file as an independent variable; all assume a fixed prompt format.

C. Instruction Tuning and System Prompts

Instruction tuning [9] and system-prompt design [10] are well-studied for conversational agents, but coding agents face a distinct regime: instructions must persist across dozens of

Fig. 1. Benchmark pipeline architecture

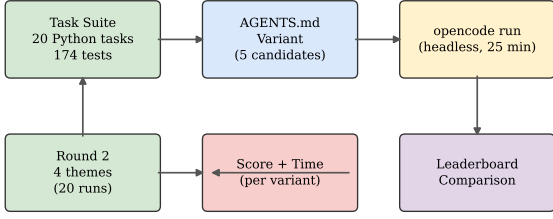


Fig. 1. Benchmark pipeline architecture.

TABLE I
AGENTS.MD VARIANTS UNDER TEST

ID	Description	Lines
none	Control: no instructions	0
concise	Minimal best-practices	18
tdd-rigorous	Test-first, verification-heavy	42
orchestrator-heavy	Multi-agent parallelism	65
user-proposed	Tiered orchestration protocol	78

tool-use turns, survive context-window truncation, and remain effective when the agent’s reasoning budget is limited by wall-clock time.

Our work is, to our knowledge, the first to isolate AGENTS.md variant as the independent variable in a controlled coding-agent benchmark.

III. BENCHMARK DESIGN

A. Architecture

Figure 1 shows the benchmark pipeline. Each run begins with a frozen task suite, applies one AGENTS.md variant, launches `opencode` run in headless mode with a 25-minute budget, and records the score (tests passed / total tests $\times$ 100%) and wall-clock time.

B. Variants Under Test

Table I describes the five instruction-file candidates. All are written in Markdown and placed at the project root as AGENTS.md.

C. Task Suites

Round 1 uses 20 self-contained Python tasks (stdlib only), each with a deliberately broken implementation and a hidden test suite totaling 174 tests. Test files are SHA-256-hashed after each run to detect agent modification.

Round 2 introduces four thematic tracks:

- **Track A** (spec-to-code): implement 8 third-party libraries from documentation; 177 hidden tests.
- **Track B** (refactor-under-constraints): $0.6\times$ behavior + $0.4\times$ structural scoring; 22 behavior + 16 structural checks.
- **Track C** (SQL analytics): 15 exact-match questions against a SQLite database.
- **Track D** (API client): 6 scenarios against a Flask mock server.

Fig. 2. Round 1 results — bug-fix suite (all variants scored 100%)

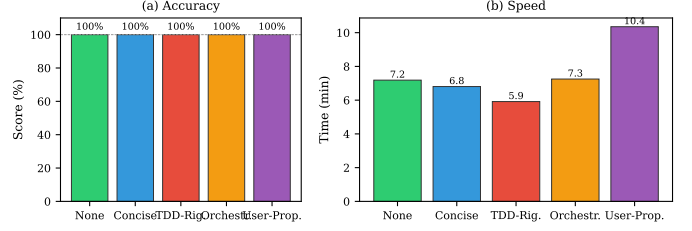


Fig. 2. Round 1 results: bug-fix suite (20 tasks, 174 tests). All variants scored 100%; only wall-clock time differed.

Fig. 3. Track A accuracy — the only track that broke saturation

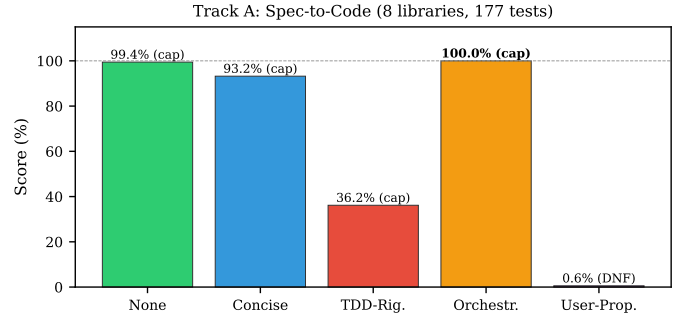


Fig. 3. Track A accuracy. Only orchestrator-heavy achieved 100%. tdd-rigorous and user-proposed paid heavy process taxes.

D. Reproducibility

All runs use the model `opencode-go/ox-alpha-free` at temperature 0. The harness isolates each run via `XD_CONFIG_HOME` overrides, suppresses non-essential stderr through `cmd /c` wrappers on Windows, and records `transcript.txt`, `score.json`, and `meta.json` per run. The full benchmark suite, all results, and the three specialist agents are published at <https://github.com/Sathvikar01/agents-md-arena>.

IV. ROUND 1: BUG-FIX SUITE

Figure 2 presents the Round 1 results. All five variants achieve 100% accuracy; the only differentiator is speed.

The `tdd-rigorous` variant completes fastest (5.9 min) because its test-first protocol eliminates wasted diagnostic turns. The `user-proposed` variant is slowest (10.4 min) due to its “understand-everything-first” analysis phase, which front-loads reading before any edits. The control (`none`) and concise variants cluster at 6.8–7.2 min, while the orchestrator variant (7.3 min) spends time spawning subagent waves that yield no benefit on these narrow tasks.

Takeaway: On simple, single-file bug fixes, instruction complexity adds overhead without improving accuracy. The control (no instructions) is competitive.

V. ROUND 2: FOUR THEMATIC TRACKS

A. Track A: Spec-to-Code (The Differentiator)

Track A is the only track that breaks saturation (Figure 3).

Fig. 4. Round 2 accuracy across all four tracks

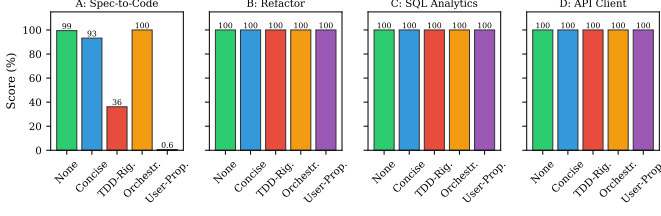


Fig. 4. Round 2 accuracy across all four tracks. Only Track A breaks saturation.

Fig. 5. Wall-clock time by variant and track

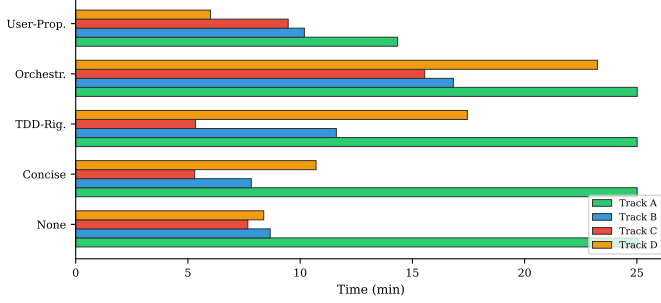


Fig. 5. Wall-clock time by variant and track.

The *orchestrator-heavy* variant is the only one to complete all 177 tests within the 25-minute budget, achieving 100%. Its strategy of spawning parallel subagent waves—one per library—allows it to process eight independent implementation tasks concurrently.

The control (*none*) scores 99.44%, missing one test at the 25-minute cap. The *concise* variant reaches 93.22%, also capped. The *tdd-rigorous* variant scores only 36.16%: its disciplined test-first cycle, while effective on narrow tasks, consumes too much of the 25-minute budget on eight independent library implementations. The *user-proposed* variant suffers four consecutive provider stream drops before any edit is applied, resulting in 0.56%—a result confounded by infrastructure failure rather than instruction quality.

B. Tracks B, C, D: Saturated

All five variants score 100% on Tracks B (refactor), C (SQL), and D (API). These tracks are too easy to differentiate instruction quality, but they reveal speed differences (Figure 4).

C. Speed Comparison

Figure 5 shows wall-clock time across all tracks. The *orchestrator-heavy* variant is consistently the slowest on saturated tracks (B/C/D) due to subagent spawn overhead, but this overhead *invests* itself in parallelism that pays off on Track A.

D. Speed–Accuracy Trade-off

Figure 6 plots average speed against average accuracy across all four Round 2 tracks. The *orchestrator-heavy* variant occupies the upper-right (slowest but most accurate), while

Fig. 6. Speed vs. accuracy trade-off (Round 2)

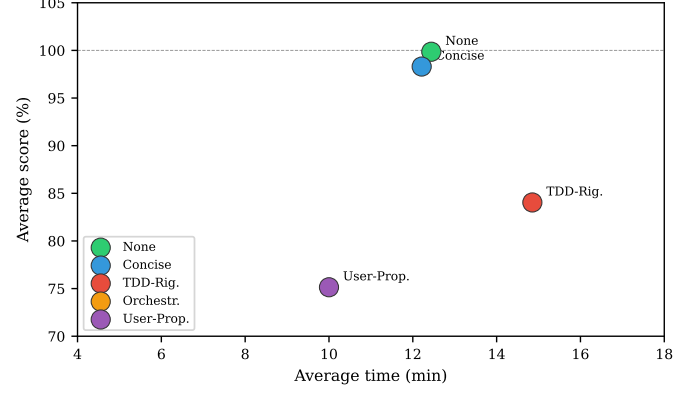


Fig. 6. Speed vs. accuracy trade-off (Round 2 averages).

Fig. 7. Variant × Track accuracy heatmap

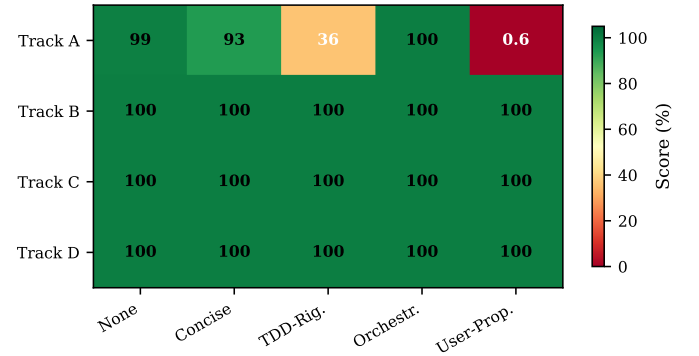


Fig. 7. Variant × Track accuracy heatmap.

the *concise* variant occupies the lower-left (fastest but less accurate on Track A).

E. Heatmap

Figure 7 presents a variant × track accuracy heatmap, making the Track A differentiation immediately visible.

F. Provider Dropout Analysis

The *user-proposed* variant’s Track A failure (0.56%) deserves close examination. Figure 8 shows the timeline: four of five prompt-response cycles are terminated by provider-level stream drops before any file edit is applied.

This failure is *not* an instruction-quality problem: the tiered protocol itself is sound, but its front-loaded analysis phase keeps the agent in long reasoning turns that are vulnerable to stream interruptions. When the provider is stable (as in Tracks B/C/D), the variant performs perfectly.

VI. DISCUSSION

A. Instruction Complexity vs. Task Breadth

Our results reveal a crossover: simple tasks (Tracks B/C/D, Round 1) are best served by minimal or no instructions, while breadth-under-deadline tasks (Track A) benefit from orchestration-heavy protocols that enable parallelism. This suggests that the optimal instruction file is *task-dependent*, not universally optimal.

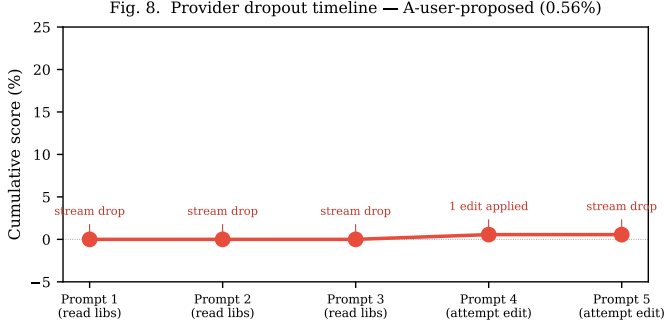


Fig. 8. Provider dropout timeline for A-user-proposed. Four of five cycles terminated by stream drops.

B. Provider Reliability as Confounding Variable

The user-proposed variant’s Track A failure demonstrates that provider reliability can dominate instruction quality. In production deployments, retry logic and provider failover are at least as important as prompt design. We recommend that future benchmarks report provider availability and stream-drop rates alongside accuracy metrics.

C. Speed–Accuracy Pareto Frontier

The orchestrator-heavy variant and the control (none) define the Pareto frontier: the former for maximum accuracy, the latter for maximum speed on narrow tasks. The `tdd-rigorous` variant, despite its theoretical appeal, falls below the frontier on breadth tasks due to excessive diagnostic overhead.

D. Lessons for Practitioners

- 1) For **narrow, single-file tasks**: use minimal instructions (or none). The control scored 99.86% overall.
- 2) For **broad, multi-library tasks**: use orchestration-heavy instructions that encourage parallel subagent use.
- 3) For **time-critical deployments**: avoid front-loaded analysis protocols; they are vulnerable to provider drops.
- 4) Always implement **retry logic**: provider stream drops are common and confound instruction quality.

VII. RELEASED ARTIFACTS

As a practical outcome of this benchmark, we release three reusable OpenCode specialist agents:

- `arena-runner`: launches and scores benchmark runs with provider-dropout retry logic and results-only commits.
- `arena-analyst`: evidence-first leaderboard aggregation with cap/DNF honesty markers.
- `suite-author`: authors new task suites under the repository’s verified integrity conventions.

These agents are installed globally via a single PowerShell script and include slash commands (`/run-arena`, `/analyze-arena`) for interactive use. They are deliberately shipped without a pinned model, as provider model IDs were observed to rotate during the experiment (`opencode-go/ox-alpha-free` vanished mid-session).

VIII. CONCLUSION

This paper presented the first empirical comparison of `AGENTS.md` instruction-file variants for LLM-based coding agents. Across 25 tasks, 4 thematic tracks, and 5 instruction variants, we found:

- 1) On narrow tasks, instruction complexity adds overhead without improving accuracy; no instructions is competitive.
- 2) On breadth-under-deadline tasks, orchestration-heavy instructions that enable parallelism are the only variant to achieve perfect accuracy.
- 3) Provider reliability is a confounding variable that can dominate instruction quality for protocols with long reasoning turns.
- 4) The speed–accuracy trade-off is real and measurable: the fastest variant (5.9 min on Round 1) is 40% slower than the slowest on Round 2 Track A, but 64% faster on Track C.

Future work should explore adaptive instruction files that adjust complexity based on task difficulty, and should include provider-failover mechanisms as a first-class experimental variable.

ACKNOWLEDGMENT

The author thanks the OpenCode development team for the open-source agent framework, and the providers behind the `ox-alpha-free` model for research access.

REFERENCES

- [1] Cursor AI, “Cursor: The AI-first code editor,” 2024. <https://cursor.sh>
- [2] P. Gauthier, “Aider: AI pair programming in your terminal,” 2024. <https://aider.chat>
- [3] OpenCode, “OpenCode: AI coding agent,” 2025. <https://opencode.ai>
- [4] C. E. Jimenez *et al.*, “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proc. ICLR*, 2024.
- [5] M. Chen *et al.*, “Evaluating large language models trained on code,” *arXiv:2107.03374*, 2021.
- [6] J. Austin *et al.*, “Program synthesis with large language models,” *arXiv:2108.07732*, 2021.
- [7] B. Rozière *et al.*, “Code Llama: Open foundation models for code,” *arXiv:2308.12950*, 2023.
- [8] GitHub, “GitHub Copilot: Your AI pair programmer,” 2023. <https://github.com/features/copilot>
- [9] L. Ouyang *et al.*, “Training language models to follow instructions with human feedback,” in *Proc. NeurIPS*, 2022.
- [10] J. Wei *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” in *Proc. NeurIPS*, 2022.